1 Introduction

Medical expenses can be a major concern for individuals and families, as costs often change depending on age, lifestyle, or health condition. Knowing these costs in advance helps people budget and make smarter choices about their health.
We frame this as a supervised regression problem: given demographic and health features, predict the billed medical insurance charges. We begin with linear regression as a baseline, then explore polynomial regression to capture non-linearities, and finally compare with an ensemble approach, random forest regression.
The report is structured as follows: Section 1 introduces the task. Section 2 formulates the problem , including dataset, data type and labels. Section 3 describes methodology, covering preprocessing, feature selection, models, hypothesis spaces, loss function and data splitting. Section 4 presents results for all models on training, validation and test sets. Finally Section 5 concludes the work and suggests improvements.

2 Problem Formulation

The dataset was obtained from Kaggle[1] and contains information about 1,338 individuals from the United States. Each entry in the dataset contains demographic and health details, including age, sex, body mass index (BMI), number of children, smoking status and region.

- **Numerical features:** age, BMI, children
- **Categorical features:** sex (male/female), smoker (yes/no), region (northeast, northwest, southeast, southwest)
- **Target variable:** charges (USD billed to the individual)

This constitutes a supervised regression task, where the goal is to learn a mapping from the given features to a continuous target (insurance charges)

3 Methods

3.1 Dataset Description

The dataset has 1,338 data points, none with missing values. Each data point has 7 columns: six are features (three numerical: age, BMI, children and three categorical: sex, smoker, region) and one is the target variable (charges).

- **Age:** ranges from 18 to 64.
- **Sex:** male/female.
- **BMI:** ranges from 16 to 53.
- **Children:** ranges from 0 to 5.
- **Smoker:** yes/no.
- **Region:** northeast, northwest, southeast, southwest.
- **Charges (target):** ranges from $1,122 to $63,770.

3.2 Preprocessing

Categorical variables are converted to numeric as follows: sex (male = 1, female = 0) and smoker (yes = 1, no = 0). Region is one-hot encoded with three dummy columns, while numerical features (age, BMI, children) are used on their original scale (no standardization). All six features are retained. This decision is guided by domain

knowledge—age, BMI, and smoking behaviour are known cost drivers—and further supported by Random Forest feature importances, which confirmed that smoker status and BMI had the largest contributions, followed by age. This makes sense from a medical standpoint, since smoking, body weight and age are well-known drivers of healthcare costs.

3.3 Models

3.3.1 Linear Regression

Linear Regression fits a linear predictor $\hat{y} = X\beta$ by minimizing squared error. It is a transparent baseline that reveals monotonic relationships between features and charges and provides a reference point for more flexible models.

3.3.2 Polynomial Regression

Polynomial regression augments the original features with squared and interaction terms via a polynomial feature map. In this work, degree 2 is used, which allows smooth non-linear effects such as quadratic BMI pattern and interactions (for example smoker x BMI) while keeping the model linear in parameters and easy to optimize with ordinary least squares. Degree 2 was selected empirically because it offered the most accurate predictions in cross-validation without excessive variance.

3.3.3 Random Forest Regressor

Random forest is an ensemble of decision trees built on bootstrap samples. Each split minimizes mean squared error, enabling the model to capture non-linearities and high-order interaction automatically and without feature scaling. It can handle outliers fairly well and makes it clear which features are most important for the predictions. Default settings are used with 100 trees for stability.

3.4 Loss Function

Linear and polynomial regression are trained by ordinary least squares, which minimizes mean squared error (MSE). Random forest grows trees by greedily minimizing MSE at each node split. These choices are standard for regression and align training objectives across methods.

3.5 Data Splitting and Validation

The dataset is split into 80% training and 20% testing with a fixed random_state=42 to ensure reproducibility and model selection is performed using 5-fold cross-validation on the training split only. The held-out test split is used exactly once at the end to report the final performance of the chosen method and to avoid information leakage.

3.6 Evaluation Metrics

Performance is quantified with mean absolute error (MAE), root mean squared error (RMSE) and coefficient of determination R^2. MAE reports the average absolute deviation in currency units, RMSE penalizes large error more strongly, and R^2 indicates the proportion of variance explained.

4 Results

4.1 Performance Evaluation Measures

All three metrics (MAE, RMSE and R^2) are reported for 5-fold cross-validation on the training data and for the held-out test split. Training errors are available in the appendix but omitted from the main table for clarity.

4.2 Empirical Outcomes

Random Forest Regressor achieved the best cross-validated performance and was selected as the final model, because its test-set metrics closely matched the cross-validation scores, indicating good generalization. Polynomial regression performed competitively, with slightly higher R^2 but higher MAE. Linear regression underfit relative to the other two models

Table 1. Training Results

Model	MAE	R^2	RMSE
Linear Regression	4208	0.742	6105
Polynomial Regression	2992	0.842	4778
Random Forest Regressor	1067	0.974	1938

Table 2. Cross Validation and Test Results

Model	CV MAE	CV R^2	CV RMSE	Test MAE	Test R^2	Test RMSE
Linear Regression	4235	0.739	6123	4181	0.784	5796
Polynomial Regression	2987	0.833	4887	2730	0.867	4551
Random Forest Regressor	2802	0.829	4947	2551	0.865	4577

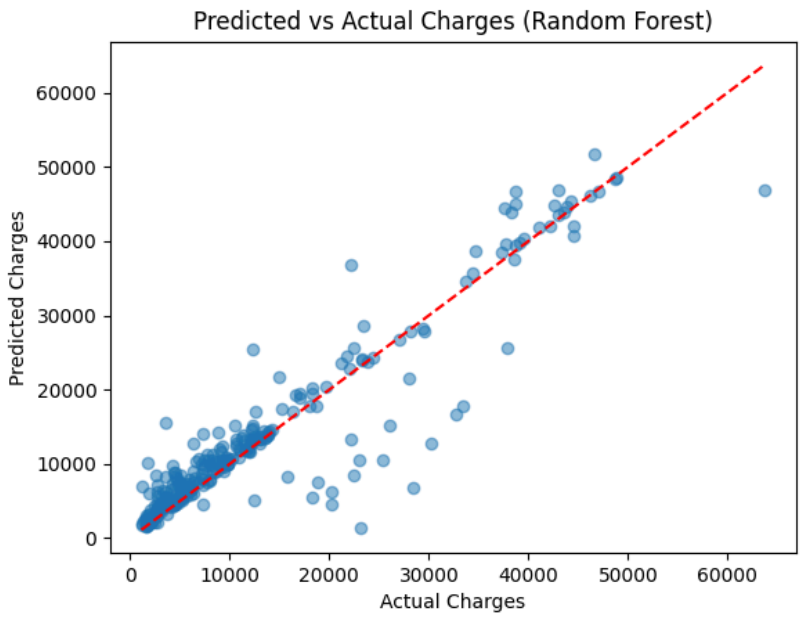

Figure 1. Predicted vs Actual Charges on the held-out test set. The dashed 45° line indicates perfect prediction.

5 Conclusion

Random Forest Regressor achieved the lowest test MAE (2551) and a strong R^2 of 0.865, making it the most effective model overall. Polynomial regression achieved slightly higher R^2 of 0.867 but with larger errors. Linear regression underfit and served as a baseline. Overall, Random Forest Regressor turned out to be the best fit for this dataset, since it gave the lowest MAE and a solid R^2, making it the most accurate model.

References:

[1] Medical Insurance Cost Dataset
https://www.kaggle.com/datasets/mosapabdelghany/medical-insurance-cost-dataset
[2] Lecture slides

Appendix:

[3]

```python
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

if __name__ == '__main__':

    df = pd.read_csv("insurance.csv")
    print(df.head())
    print(df.info())
    print(df.describe())

    # Encode categorical
    df['sex'] = df['sex'].map({'male': 1, 'female': 0})
    df['smoker'] = df['smoker'].map({'yes': 1, 'no': 0})
    df = pd.get_dummies(df, columns=['region'], drop_first=True)

    X = df.drop("charges", axis=1)
    y = df["charges"]

    X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42)
```

```python
    # Models
    linear = LinearRegression()
    poly = make_pipeline(PolynomialFeatures(degree=2, include_bias=False), LinearRegression())
    rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)

    models = {"Linear": linear, "Polynomial (deg=2)": poly, "Random Forest": rf}

    cv = KFold(n_splits=5, shuffle=True, random_state=42)

    results_train = []
    results_cv = []

    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred_train = model.predict(X_train)
        train_mae = mean_absolute_error(y_train, y_pred_train)
        train_rmse = np.sqrt(mean_squared_error(y_train, y_pred_train))
        train_r2 = r2_score(y_train, y_pred_train)

        cv_mae = -cross_val_score(model, X_train, y_train, scoring='neg_mean_absolute_error', cv=cv).mean()
        cv_rmse = -cross_val_score(model, X_train, y_train, scoring='neg_root_mean_squared_error', cv=cv).mean()
        cv_r2 = cross_val_score(model, X_train, y_train, scoring='r2', cv=cv).mean()

        results_train.append([name, train_mae, train_rmse, train_r2])
        results_cv.append([name, cv_mae, cv_rmse, cv_r2])

    result_train_df = pd.DataFrame(results_train, columns=["Model", "Train MAE", "Train RMSE", "Train R2"])
    result_df = pd.DataFrame(results_cv, columns=["Model", "CV MAE", "CV RMSE", "CV R2"])
    print("\nTraining results:\n", result_train_df)
    print("\nCross-validation results:\n", result_df)

    # Evaluate each model on the test set
    results_test_set = []
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred_test = model.predict(X_test)
        test_mae = mean_absolute_error(y_test, y_pred_test)
        test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
        test_r2 = r2_score(y_test, y_pred_test)
        results_test_set.append([name, test_mae, test_rmse, test_r2])

    result_test_set_df = pd.DataFrame(results_test_set, columns=["Model", "Test MAE", "Test RMSE", "Test R2"])
    print("\nTest results:\n", result_test_set_df)
```

```python
# Select best (by CV MAE)
    best = result_df.loc[result_df["CV MAE"].idxmin(), "Model"]
    print("\nBest model by CV MAE:", best)

    final_model = models[best]
    final_model.fit(X_train, y_train)
    y_pred = final_model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print("\nFinal test performance:")
    print("MAE:", round(mae, 2))
    print("RMSE:", round(rmse, 2))
    print("R²:", round(r2, 3))

    # If the best model is Random Forest, print feature importances
    if best == "Random Forest":
        importances = pd.Series(final_model.feature_importances_, index=X_train.columns)
        print("\nRandom Forest feature importances:")
        print(importances.sort_values(ascending=False).head(5))

    # Plot predicted vs actual
    plt.scatter(y_test, y_pred, alpha=0.5)
    plt.xlabel("Actual Charges")
    plt.ylabel("Predicted Charges")
    plt.title(f"Predicted vs Actual Charges ({best})")
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    plt.show()
```

Table 3. Random Forest Feature Importances (Top 5)

Feature	Importance
Smoker	0.61
BMI	0.22
Age	0.13
Children	0.02
Sex	0.01